



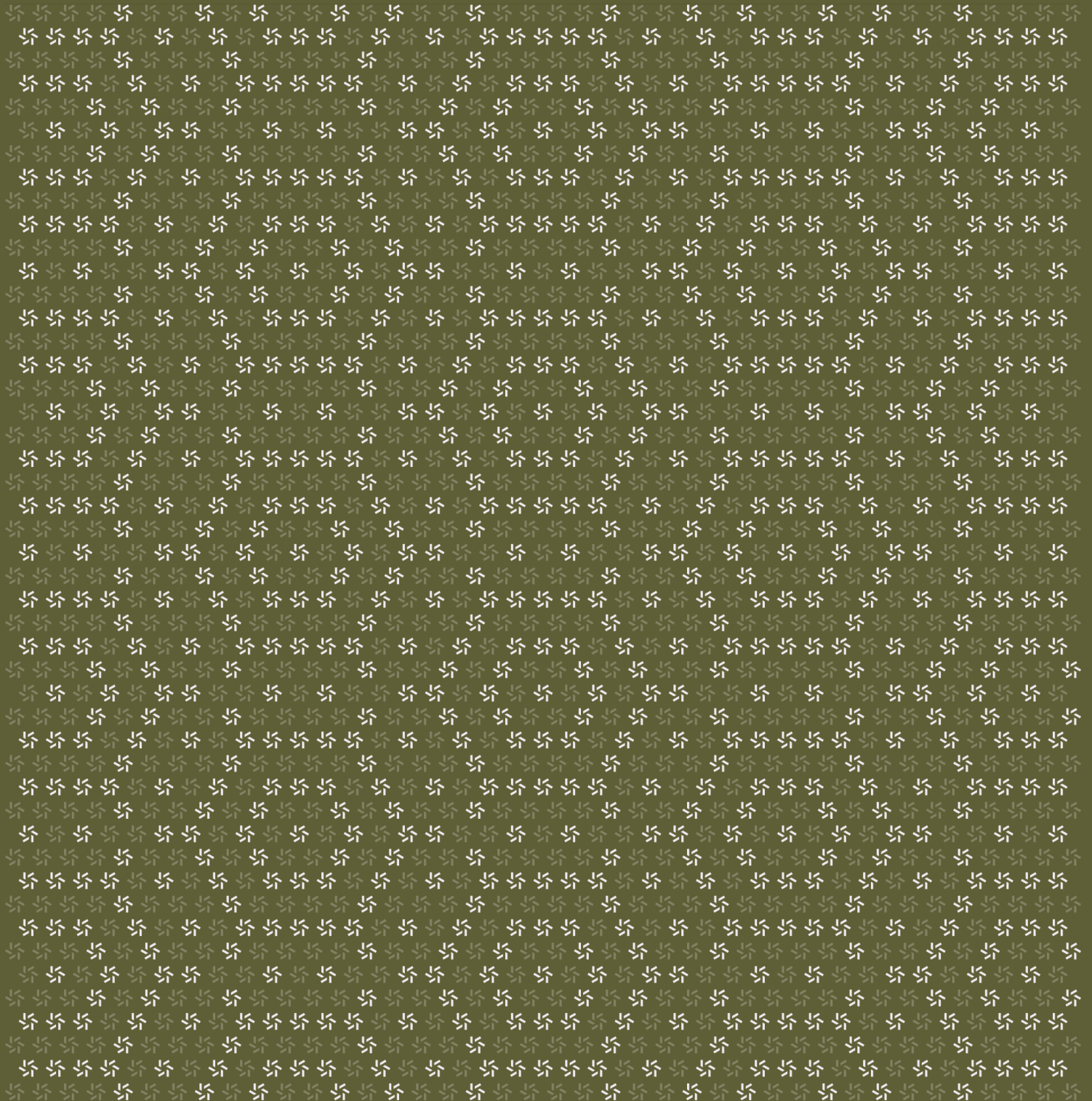
Prepared for
Julien Genestoux
Uneven Labs, Inc.

Prepared by
Bradán Beaver
Jinseo Kim
Zellic

April 15, 2026

Relay Protocol Oracle

Web Application Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About Relay Protocol Oracle	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Solana VM and Sui VM attestors attest reverted deposit transactions	11
3.2. Sui VM attestor does not check the origin of DepositEvent	12
3.3. Too short finalization/confirmation window is used for Ethereum VM and Bitcoin VM attestors	13
3.4. Ethereum VM, Tron VM, and Bitcoin VM attestors do not ensure finalization/ confirmation in the getDepositoryWithdrawalMessage function	15
3.5. Multiple attestors incorrectly check orderId in the getSolverPaidAmount function	16
3.6. Sui VM attestor does not check the origin and sender of MemoEvent	17

3.7.	The <code>verifySolverCalls</code> function does not check the order of calls	18
3.8.	Sui VM attester does not properly process multiple deposits in one transaction	20
3.9.	Loss of precision via mishandled currency type	22
3.10.	Bitcoin VM attester does not check withdrawal amount and address	23

4.	Discussion	23
4.1.	Some ERC-20 tokens may not be compatible	24
4.2.	False positive on order-ID heuristic may result in temporary lock of funds	24

5.	System Design	24
5.1.	Component: Oracle	25

6.	Assessment Results	27
6.1.	Disclaimer	29

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Uneven Labs, Inc. from March 10th to March 23rd, 2026. During this engagement, Zellic reviewed Relay Protocol Oracle's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can the oracle be tricked into producing a valid attestation for an event that did not actually happen on chain?
 - Are there ways to replay attestations to trigger duplicate mints/transfers on the Hub?
 - Is the peer-to-peer oracle communication secure against forged or manipulated responses from compromised peers?
 - Can a malicious RPC endpoint cause the oracle to produce incorrect attestations?
 - Are the HTTP endpoints properly authenticated and protected against unauthorized requests?
 - Is it possible for a single compromised oracle instance to unilaterally settle funds?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Underlying infrastructure relating to hosting oracles
- Related on-chain contracts

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Relay Protocol Oracle files, we discovered 10 findings. Two critical issues were found. Three were of high impact, three were of medium impact, and one was of low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of

Uneven Labs, Inc. in the Discussion section ([4](#), [7](#)).

Based on the number and severity of findings identified during this engagement, we recommend a comprehensive reassessment of the codebase following remediation efforts; see section [6](#), [7](#)

Breakdown of Finding Impacts

Impact Level	Count
Critical	2
High	3
Medium	3
Low	1
Informational	0
Indeterminate	1



2. Introduction

2.1. About Relay Protocol Oracle

Uneven Labs, Inc. contributed the following description of Relay Protocol Oracle:

Relay is interoperability infrastructure for the onchain money stack — a decentralized protocol that connects wallets, payment processors, and blockchains. It sits at the interoperability layer between orchestration (payment processors) and settlement (blockchains), acting as neutral infrastructure that enables value to move seamlessly across different chains. Relay is not a wallet, processor, or blockchain itself — it's the connective tissue that allows users and applications to transact across the fragmented onchain ecosystem without being tied to any particular interface or settlement layer.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Unsafe code patterns. Many vulnerabilities in web-based applications arise from oversights during development. Missing an authentication check on a single API endpoint can critically impact the security of the overall application. Failure to properly sanitize and encode user input can lead to vulnerabilities like SQL injection, server-side request forgery, and more. We use both automated tools and extensive manual review to identify unsafe code patterns.

Business logic errors. Business logic is the heart of all web applications. We carefully review logic to ensure that the code securely and correctly implements the specified functionality. We also thoroughly examine all specifications for inconsistencies, flaws, and vulnerabilities, including for risks of fraud and abuse.

Integration risks. Web projects frequently have many third-party dependencies and interact with services and libraries that are not under the developer's control. We review the project's external interactions and identify potentially dangerous behavior resulting from compatibility issues and data inconsistencies.

Code maturity. We look for possible improvements across the entire codebase, reviewing violations of industry best practices and code quality standards. We suggest improvements to code clarity, documentation, and testing practices.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped files itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Relay Protocol Oracle Files

Type	TypeScript
Platform	Node
Target	relay-protocol-oracle
Repository	https://github.com/relayprotocol/relay-protocol-oracle ↗
Version	7dbe806a9ee9bbc0f32df8bc7e0b6da1de253799
Programs	src/**/*.ts

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.4 person-weeks. The assessment was conducted by two consultants over the course of two calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Bradan Beaver
✈ Engineer
bradan@zellic.io ↗

Jinseo Kim
✈ Engineer
jinseo@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 10, 2026 Start of primary review period

March 12, 2026 Kick-off call

March 23, 2026 End of primary review period

The originally planned end date of the review period was March 19, but it was extended to March 23 to accommodate for the postponed kick-off call.

3. Detailed Findings

3.1. Solana VM and Sui VM attestors attest reverted deposit transactions

Target	SolanaVmAttestor, SuiVmAttestor		
Category	Coding Mistakes	Severity	Critical
Likelihood	High	Impact	Critical

Description

The `getDepositoryDepositMessages` function of an attestor processes the attestation request of a deposit to the depository by a user. A deposit is linked to an order, and a solver may fill the order, which would increase the Hub balance that can be withdrawn from the depository.

An attestor must not attest a reverted deposit transaction. However, attestors of Solana VM and Sui VM networks do not ensure that a given transaction is not reverted during execution.

Impact

A malicious user may send a deposit transaction that always reverts in order to steal assets from the depository.

Recommendations

Consider ensuring that a given deposit transaction is not reverted in the `getDepositoryDepositMessages` function.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in commit [7f973aa8](#).

Uneven Labs, Inc. provided the following notes regarding this fix:

The Solana VM part of the code was fixed, while for Sui VM we decided to remove support completely - the Sui VM code was never used in production and will be removed from the codebase.

3.2. Sui VM attester does not check the origin of DepositEvent

Target	SuiVmAttestor		
Category	Coding Mistakes	Severity	Critical
Likelihood	High	Impact	Critical

Description

The `getDepositoryDepositMessages` function of an attester processes the attestation request of a deposit to the depository by a user. A deposit is linked to an order, and a solver may fill the order, which would increase the Hub balance that can be withdrawn from the depository.

The attester of the Sui VM network does not ensure that `DepositEvent`, which is emitted by the depository contract after a user deposits their asset to the depository, is from the depository.

Impact

A malicious user may send a transaction that emits `DepositEvent` without depositing to the depository in order to steal assets from the depository.

Recommendations

Consider ensuring that `DepositEvent` from the given deposit transaction is emitted by the depository.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

Uneven Labs, Inc. noted that the Sui VM code will be removed from the codebase.

3.3. Too short finalization/confirmation window is used for Ethereum VM and Bitcoin VM attestors

Target	EthereumVmAttestor, BitcoinVmAttestor		
Category	Business Logic	Severity	High
Likelihood	Medium	Impact	High

Description

When off-chain infrastructures for certain networks handle a transaction on chain, they generally have to wait until the transaction is finalized or confirmed (i.e., it is unlikely that the network is reorganized in a way that does not include the transaction).

The Ethereum VM attestor assumes a transaction is finalized after 60 seconds:

```
export class EthereumVmAttestor extends VmAttestor {
  // (...)
  private _FINALIZATION_TIME = 60n;

  private async _ensureTxFinalization(
    chainId: string,
    tx: TransactionReceipt,
    trackingId: string,
  ) {
    // (...)
    const latestBlockTimestamp = await rpc.getBlock().then((b) => b.timestamp);
    // (...)
    const txTimestamp = await rpc
      .getBlock({ blockNumber: tx.blockNumber })
      .then((b) => b.timestamp);
    if (latestBlockTimestamp - txTimestamp < this._FINALIZATION_TIME) {
      throw externalError(`Transaction ${tx.transactionHash} is not finalized`);
    }

    return txTimestamp;
  }
}
```

This period is equivalent to five blocks, which is too short considering that the Ethereum network experienced a seven-block reorg in 2022.

The Bitcoin VM attester assumes a transaction is confirmed after two blocks:

```
export class BitcoinVmAttester extends VmAttester {  
  // (...)  
  private _FINALIZATION_BLOCKS = 2;  
  
  private _ensureTxFinalization(  
    transactionId: string,  
    tx: { confirmations?: number },  
  ) {  
    if (!tx.confirmations || tx.confirmations < this._FINALIZATION_BLOCKS) {  
      throw externalError(`Transaction ${transactionId} is not finalized`);  
    }  
  }  
  // (...)  
}
```

This is also too short. Bitcoin Core uses six blocks as the default threshold, and there have been multiple two-block reorgs in the Bitcoin network.

Additionally, the Ethereum VM attester can be used in any EVM-compatible network, but the finalization condition is hardcoded as 60 seconds in the Ethereum VM attester.

Impact

An attested transaction can be excluded from the network later due to a chain reorg, which would result in loss of funds in the depository.

Recommendations

Consider attesting a transaction only after it is sufficiently finalized or confirmed. For example, blocks marked as "finalized" can be safely attested in Ethereum chain (please note that this can be different in EVM-compatible chain), and six blocks can be appropriate threshold for accepting a block as confirmed in Bitcoin chain.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and fixes were implemented in the following commits:

- [18a05acf](#) ↗
- [c52198f1](#) ↗
- [35ad4cde](#) ↗
- [ea8f995c](#) ↗

3.4. Ethereum VM, Tron VM, and Bitcoin VM attestors do not ensure finalization/confirmation in the `getDepositoryWithdrawalMessage` function

Target	EthereumVmAttestor, TronVmAttestor, BitcoinVmAttestor		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

The `getDepositoryWithdrawalMessages` function of an attestor processes the attestation request of a withdrawal to the depository by a solver. This finalizes the decrease of the Hub balance of the solver.

The `getDepositoryWithdrawalMessages` functions in the Ethereum VM attestor, Tron VM attestor, and Bitcoin VM attestor do not ensure that a given transaction is finalized or confirmed.

Impact

If a chain is reorganized in a way that does not include a withdrawal transaction after the oracle attests the withdrawal transaction, a solver will not be able to receive withdrawn assets and their Hub balance will not be recovered.

Recommendations

Consider attesting a transaction only after it is sufficiently finalized or confirmed. For example, blocks marked as "finalized" can be safely attested in Ethereum chain (please note that this can be different in EVM-compatible chain), and six blocks can be appropriate threshold for accepting a block as confirmed in Bitcoin chain.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and fixes were implemented in the following commits:

- [f76e7af0](#) ↗
- [6ec03874](#) ↗

3.5. Multiple attestors incorrectly check orderId in the getSolverPaidAmount function

Target	LighterVmAttestor, HyperliquidVmAttestor		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

The `getSolverPaidAmount` function of an attestor attests that a solver paid the assets to the destination as part of processing an order. The solver submits this attestation to the Hub chain to have the source assets of the order unlocked as the balance of the Hub chain.

The `getSolverPaidAmount` functions in the Lighter VM and Hyperliquid VM attestors do not check the `orderId` of a transaction. Solana VM, Bitcoin VM, and Sui VM check the `orderId`, but in the case a transaction includes multiple `orderIds`, they will attest all payments of all orders with the transaction.

Impact

Attestors can incorrectly attest solver payments for unrelated orders, allowing for replay or cross-order payment hijacking when a solver submits a valid transfer that is later reused for a different order.

Recommendations

Consider ensuring that `orderId` is checked and validated within `getSolverPaidAmount` functions.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and fixes were implemented in the following commits:

- [8eb24981](#) ↗
- [5395289f](#) ↗

3.6. Sui VM attester does not check the origin and sender of MemoEvent

Target	SuiVmAttestor		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	Medium

Description

The Sui VM attester expects a solver to leave MemoEvent with the orderId in their transaction sending assets to the destination. However, it does not check who emitted the MemoEvent.

Additionally, it seems the expected emitter of MemoEvent emits the MemoEvent when anyone requests — it is not checked who requested to emit the MemoEvent being processed.

Impact

If a solver mistakenly invokes a malicious contract, it may emit the MemoEvent to steal the payment transaction (i.e., the transaction will not be regarded as the payment from the actual solver, assuming that Finding [3.5.7](#) is resolved).

Recommendations

Consider validating the origin and sender of MemoEvent.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

Uneven Labs, Inc. noted that the Sui VM code will be removed from the codebase.

3.7. The verifySolverCalls function does not check the order of calls

Target	EthereumVmAttestor		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

An order towards an EVM-compatible chain may specify the calls that a solver has to make, which will be verified by the oracle. The Ethereum VM attestor implements the `verifySolverCalls` function, which is responsible for checking the calls:

```
export class EthereumVmAttestor extends VmAttestor {
  // (...)
  public async verifySolverCalls(
    chainId: string,
    transactionId: string,
    calls: string[],
    extraData: string,
  ): Promise<boolean> {
    // (...)
    const parsedLogs = parseEventLogs({
      abi: ABI,
      logs: receipt.logs,
      eventName: ["SolverCallExecuted"],
    }).filter((log) => {
      if (
        log.eventName === "SolverCallExecuted" &&
        log.address.toLowerCase() === fillContract.toLowerCase()
      ) {
        return true;
      }
      return false;
    });

    const usedParsedLogIndexes = new Set<number>();
    for (const call of calls) {
      const decodedCall = decodeOrderCall(call, chain.vmType);

      const relevantLogIndex = parsedLogs.findIndex(
        (log, i) =>
```

```

        log.args.to.toLowerCase() === decodedCall.call.to.toLowerCase() &&
        log.args.data.toLowerCase() === decodedCall.call.data.toLowerCase()
    &&
        log.args.amount === BigInt(decodedCall.call.value) &&
        !usedParsedLogIndexes.has(i),
    );
    if (relevantLogIndex === -1) {
        return false;
    } else {
        usedParsedLogIndexes.add(relevantLogIndex);
    }
}

return true;
}
// (...)
}

```

This function checks that all of the given calls are executed through the fill contract. However, it does not ensure that the calls are executed in the specified order.

Impact

A solver may reorder the calls in a way a user does not prefer. The severity would be up to the output calls the user requested: the call affected by the state transition of previous calls could be affected.

Recommendations

Consider checking the order the calls are executed in.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in commit [6fa5e950](#).

3.8. Sui VM attester does not properly process multiple deposits in one transaction

Target	SuiVmAttestor		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The `getDepositoryDepositMessages` function in the Sui VM attester processes the `DepositEvents` in the transaction:

```
export class SuiVmAttestor extends VmAttestor {
  public async getDepositoryDepositMessages(
    chainId: string,
    transactionId: string
  ): Promise<EnhancedDepositoryDepositMessage[]> {
    // (...)

    const messages: EnhancedDepositoryDepositMessage[] = [];

    let messageIndex = 0;
    for (const event of transaction.events) {
      const onchainId = getDeterministicId(
        chainId,
        transactionId,
        messageIndex.toString()
      );

      if (event.type.includes("DepositEvent")) {
        const typedEvent = event.parsedJson as {
          /* (...) */
        };

        messages.push({
          data: {
            chainId,
            transactionId,
          },
          result: {
            onchainId,
```

```
        depository,  
        depositId:  
            "0x" + Buffer.from(typedEvent.deposit_id).toString("hex"),  
        depository: typedEvent.from,  
        currency: typedEvent.coin_type.name,  
        amount: typedEvent.amount.toString(),  
    },  
    extraData: {  
        timestamp: transaction.timestampMs,  
    },  
    });  
    }  
    }  
    }  
    return messages;  
    }  
    // (...)  
    }
```

Note that the `onchainId` will be same for all messages in the same transaction since `messageIndex` is never incremented.

Impact

The `onchainId`, which is the deterministic identifier of a deposit, would be same if the messages are in the same transaction, which may result in denial of the deposit to be processed by the Hub chain.

Recommendations

Consider incrementing the `messageIndex` over the loop in the `getDepositoryDepositMessages` function of the Sui VM attester.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

Uneven Labs, Inc. noted that the Sui VM code will be removed from the codebase.

3.9. Loss of precision via mishandled currency type

Target	HyperliquidVmAttestor		
Category	Coding Mistakes	Severity	Low
Likelihood	High	Impact	Low

Description

Hyperliquid attestation pipelines handle currencies as JavaScript type `Number`, introducing precision loss. If the decimal count is large enough (e.g., 18), the value cannot be represented exactly, causing rounded values to pass or fail incorrectly.

Impact

Rounded-down values can understate payments or deposits, letting insufficient transfers pass attestation or producing incorrect settlement amounts. However, Hyperliquid contains limited precision irregardless and may still lead to precision errors.

Recommendations

Keep on-chain amounts as `String` or `BigInt`, avoiding `Number`, and feed `String` / `BigInt` directly into precise parsers.

Remediation

This issue has been acknowledged by Uneven Labs, Inc., and a fix was implemented in commit [870f0989](#).

3.10. Bitcoin VM attester does not check withdrawal amount and address

Target	BitcoinVmAttestor		
Category	Coding Mistakes	Severity	Indeterminate
Likelihood	Low	Impact	Indeterminate

Description

The `getDepositoryWithdrawalMessages` function in the Bitcoin VM attester attests the withdrawal without verifying the actual withdrawn amount and address on chain.

Impact

We could not determine the behavior of the Hub chain when it receives the final withdrawal attestation, which does not match with the initial withdrawal request that includes the actual withdrawn amount.

Recommendations

Consider attesting the withdrawal only if the withdrawn amount and address are validated.

Remediation

This issue has been acknowledged by Uneven Labs, Inc..

Uneven Labs, Inc. provided the following notes regarding this finding:

The Bitcoin transaction data for any withdrawal is generated by the [BitcoinPayloadBuilder](#) - which is assumed to be constructing the transaction in a way consistent with the withdrawn amount/address.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Some ERC-20 tokens may not be compatible

In order to allow users to deposit assets without approving the depository beforehand, the Ethereum VM attester checks ERC-20 Transfer events as well as RelayErc20Deposit events.

Due to the business-logic handling of both of these events (as of the time of writing), ERC-20 tokens that 1) emit multiple Transfer events or 2) reenter when a user transfers tokens may not be compatible with the project.

Solvers are responsible only for accepting compatible tokens.

4.2. False positive on order-ID heuristic may result in temporary lock of funds

The `findTransfersInCalldata` function in the `getDepositoryDepositMessages` of the Ethereum VM attester employs some heuristics to figure out the order ID. One heuristic is to find all possible `transfer / transferFrom`-format input from the `calldata`, instead of the `calldata` that is actually `transfer / transferFrom`. This allows the attester to find the order ID from the `calldata` of smart contract-based wallets, which would not start with the function signature of `transfer / transferFrom`. If the following 32-byte `calldata` is nonzero, Ethereum VM attester will assume it as the order ID.

However, this heuristic could be fragile. For example, the `calldata` to the smart contract-based wallet may contain the transaction signature; it could be mistakenly recognized as the order ID, which would result in the lock of deposited funds. In this case, the user will be able to recover the funds after a default expiration window of seven days.

This issue has been acknowledged by Uneven Labs, Inc.

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: Oracle

Description

The Relay Protocol Oracle is an off-chain backend service that witnesses cross-chain activity and produces signed EIP-712 attestations for the Hub on the Relay Chain. It exposes a Fastify HTTP API that verifies deposits, solver fills, solver refunds, and withdrawal flows across multiple VM types and then returns execution or payload messages for callers to submit on chain. The code includes chain-specific VM and verification logic, peer signature fan-out, and multiple signing backends.

The service is provided through a Fastify API. Input contracts use TypeBox schemas; attestation endpoints call into `src/services/attestation/**/*`, which orchestrates chain-specific attestors to pull on-chain state, verify deposits/fills/refunds/withdrawal steps, compute deterministic idempotency keys, and build Hub actions (MINT / TRANSFER / BURN). Signing is abstracted behind `src/signers/**/*` (configurable to raw ECDSA via Viem or AWS KMS). Peer fan-out uses Axios and only accepts signatures whose execution/payload matches local computation. The service is pull-based: HTTP → validate → attest (read chain state, compute EIP-712 payloads) → sign → return for on-chain verification by the Relay Oracle contract.

Specifically, the API exposes multiple routes: swagger documentation at `/documentation`, a health check at `/lives/v1`, and a chain metadata route at `/chains/v1`. Multiple attestation routes are also exposed behind API key authentication, including

- `/attestations/depositary-deposits/v1` returning signed deposit messages and a Hub MINT execution;
- `/attestations/depositary-withdrawals/v1`, which verifies withdrawal status;
- `/attestations/withdrawal-initiation/{v1|v2}`, which computes the deterministic withdrawal address and returns a Hub TRANSFER execution to fund it;
- `/attestations/withdrawal-initiated/{v1|v2}`, which returns withdrawal payload parameters after confirming an address is funded; and
- `/attestations/solver-fills/v1`/`/attestations/solver-refunds/v1`, which validates fills/refunds against orders.

The service can optionally fan out these requests to peers for extra signatures, but only signatures matching the locally computed execution or payload are accepted in responses.

Invariants

- A server boot fails if `ENVIRONMENT` is missing or if the selected signing module lacks its required secrets — `ECDSA_PRIVATE_KEY` for raw or `AWS_KMS_SIGNER_KEY_ID / REGION` for AWS KMS.
- The `errorWrapper` returns `400 Bad Request` only for `externalError` cases; all other errors log full context and surface as a generic `500 Internal Server Error`.
- Peer fan-out never blocks peer errors, time-outs are logged and skipped, and shadow peers are excluded from aggregation.
- Peer execution signatures are accepted only if `areExecutionsEqual` is true and there are matching `idempotencyKey` and ordered actions; peer payload signatures are accepted only if `arePayloadParamsEqual` is true.
- Withdrawal-address derivation requires a configured depository for the chain and is deterministic over (depository, VM type, recipient, currency, withdrawer alias, nonce).
- Withdrawn amounts must be greater than zero; Hub balances must cover the requested amount or the request fails.
- Depository withdrawals emit `BURN` when the status is `EXECUTED`, and `TRANSFER` back when the status is `EXPIRED`; idempotency keys include withdrawal ID/transaction (and status for expired) to disambiguate.
- Solver orders must have a valid solver signature, input `onchainIds` must be unique, and each input must map to a depository deposit with matching `orderId` and `onchainId` or the request fails.
- Auth gating is binary. When the `API_KEYS` environment variable is set, all routes except `/`, `/documentation`, and `/lives/v1` require a valid `x-api-key`; when unset, all routes are open.

Test coverage

Cases covered

- Behavior of `AttestationService` — deposit attestations, withdrawal executions, solver fills, and idempotency-key derivations
- Hub-facing balance lookups and transfer-to-withdrawal-address behaviors
- Chain-specific attestation flows and deposit/fill/refund/withdrawal parsing

Cases not covered

- API-level tests (routes, prehandlers, and `errorWrapper` behavior)
- Authentication and API-key validation
- Peer fan-out behavior and signature matching
- Signing backend flow
- Load testing and concurrency issues
- End-to-end flows and HTTP request to signed response

Attack surface

Query parameters and request bodies. Callers control JSON payloads and request path parameters for attestation requests (orders, transaction hashes, withdrawal parameters, optional hints). These inputs feed chain queries and signature generation. Schema validation filters validate the format of the inputs, but correctness relies on downstream attester checks.

RPC and chain data. The service trusts chain RPC responses and event/balance data to verify deposits, fills, refunds, and withdrawals. Malicious or stale RPC providers could skew attestations, and chain VM-type-specific parsers interpret logs/receipts/balances.

Peer responses. Peer fan-out posts the same request to configured peers and accepts signatures only when the execution/payload matches exactly. Peer time-outs/errors are ignored. Peer URLs and API keys are operator-configured, and shadow peers are excluded from aggregation.

6. Assessment Results

During our assessment on the scoped Relay Protocol Oracle files, we discovered 10 findings. Two critical issues were found. Three were of high impact, three were of medium impact, and one was of low impact.

Based on the scope of findings identified during this engagement, we recommend a comprehensive reassessment of the codebase following remediation efforts. This is particularly important given that we observed multiple reoccurring patterns resulting in serious issues from the attestation system of the oracle, which is essential and critical infrastructure of the protocol.

A follow-up assessment would provide greater confidence in the overall security posture of the protocol and help ensure that remediation efforts have not introduced new issues.

To maximize the value of a follow-up engagement, we recommend first addressing the following items:

1. Documenting and testing assumptions and requirements

We recommend to materialize and document the VM-neutral assumptions and requirements of each attestation and write tests based on them. The project implements seven different attester implementations per VM. Some issues we found only existed in certain implementations, while other implementations had code explicitly preventing the issue. Organizing the assumptions and requirements may help avoid similar issues in the future and increase confidence that the implementation of new attestors will not result in vulnerabilities.

Moreover, it would be beneficial to be aware of the implicit guarantee of each chain. For example, RPCs of some VMs may not provide parseable information for reverted transactions while others may.

2. Expanding testing for malicious/uncommon inputs and negative tests

The project is well-developed and includes happy-path tests, but further review and tests concerning how the codebase handles malicious/corrupted/uncommon inputs is recommended. Most issues we discovered involved the way such inputs could break the business logic.

Consider expanding negative tests. Ideally, negative tests should not be written by verifying the failing paths of the code one by one; instead, they should be written by enumerating a number of possible incorrect inputs, anticipating outcomes possible from malicious behavior, and ensuring that the code always properly handles these cases.

3. Improving the development process

As a longer-term investment in code quality, we encourage continuing to strengthen security-focused development workflows, including:

- Comprehensive test coverage with end-to-end tests that exercise critical paths and edge cases

- Clear documentation of security assumptions, invariants, and known limitations
- Structured code review processes that explicitly consider security implications

Such measures could help identify issues like Findings [3.4](#), [7](#) and [3.8](#), [7](#) earlier in the development cycle, reducing the likelihood of small oversights leading to functional issues.

A reassessment following these improvements would allow for deeper analysis and provide greater assurance for production deployment.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.